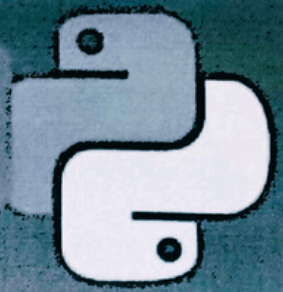
PYTHON NOTES

PYTHON MINDMAP



BASICS

- Syntax
- Variables
- Data Types
- Control Structures

FUNCTIONS

- Built-in Functions
- Defining
- Functions
- Recursion

DATA STRUCTURES

- Lists
- Tuples
- Dictionaries
- Sets

OBJECT-ORIENTED PROGRAMMING (OOP)

- Classes & Objects
- Inheritance
- Encapsulation
- Polymorphism

FILE HANDLING

- Reading with CSV
- Working with JSON

MODULES AND PACKAGES

- Importing Modules
- Creating Packages
- Using Packages

LIBRARIES AND FRAMEWORKS

- NumPy
- Pandas
- Matpiotillib
- TensorFlow
- Flask

CONCURRENCY

- Threading
- Multiprocessing
- Asyncio

WEB DEVELOPMENT

- Django
- Flask

DATA SCIENCE

- Pandas
- NumPy
- Matplotiib
- Scikit-learn
- TensorFlow

TOOLS AND IDEs

- PyCharm
- Jupyter Notebook
- Visual Studio Code
- Anaconda

TOOLS AND IDEs

- PyCharm
- Jupyter Notebook
- Visual Studio Code
- Anaconda
- Plp

|| PYTHON ||

What is Python?

⇨ Python is a beginner-friendly, high-level, general purpose and Object oriented programming language.

Why should we learn Python?

⇨

- Easy to use & learn.
- Expressive language.
- Interpreted language.
- Object-oriented language.
- Open-source language.
- Extensible.
- Dynamic memory allocation.
- GUI Programming support.

Where is Python used?

⇨

- Data Science.
- Data Mining
- Desktop Application
- Mobile Application
- Web Application
- Machine learning
- Artificial Intelligence
- Speech Recognisation

Python Popular frameworks & libraries.

1. **Web Development**: Django, Flask, Pyramid, cherrypy.
2. **GUIs based Applications**: TK, PyGUI, PyQT, PyJs.
3. **Machine learning**: TensorFlow, PyTorch, Scikit-learn.
4. **Mathematics**: NumPy, Pandas.

First Program:

```
Print ("Hello World")
```

output
Hello World

- As a integer

```
Print (2)
```

output
2

- As a storing

```
Print ('2')
```

output
2

Addition, Multiplication, division, subtraction:

```
Print (2+3)
```

output
5

```
print (2*3)
```

output
6

```
print (2/3)
```

output
0.6

```
print (2-3)
```

output
-1

NOTE

We can print it only in one line by print() function.

Comments in Python:

Symbol: #

NOTE: Comments are used to explain codes, make it readable, and help other developers to understand the code.

1.
```
# Printing helloworld
print ("Hello world")
```

2.
```
# Addition of two nums
print (2+3)
```

3.
```
# Multiplication of two nums
print (2*3)
```

① Single-line Comment

This is called single line comment.

② Multiline Comment

''' This is called
multiline comment '''

```
y = 10
print(y)
```

▲ NOTE : These are technically multiline strings, but they can act like comments if not assigned to a variable or used in a code

CHARECTER SET in ly :

1. Letters :-
   - lower case : a → z
   - Upper case : A → Z
2. Digits :-
   - 0 → 9
3. Special charecters :-
   @, #, $, %, &, *, ( ), { }, [ ], :, ; . etc
4. White space charecters :- / Escape Sequence charecters :-
   (i) Space (' ')
   (ii) Tab (\t)
   (iii) Newline (\n)
   (iv) Carriage Return (\r)
   (v) Form Feed (\f)
   (vi) Vertical Tab (\v)

(i) **Space (' ')**

```
Print ("Hello  World)
```

Output :-
Hello World

(ii) **Tab (\t)**

```
print ("Arijit \t Das")
```

output :-
Arijit Das

(iii) **Newline (\n)**

```
print ("I am a \n Student")
```

output :-
I am a
Student

(iv) Carriage return (\r)

Same as New line (\n)

Variables

A Variable in python [in any programming language] is a name given by the user which are located in memory. Variable helps to store data types. [Integer, string, float, bool] and other data structures [list, dictionary, tuple]. Basically it stores the data.

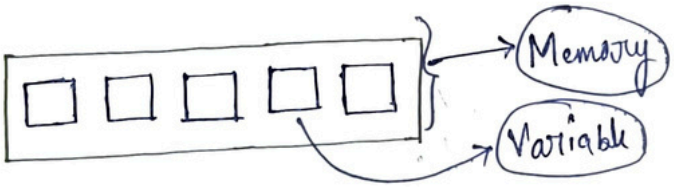
(*) Example :-

name = "Arijit" → Here name is the variable which stores the string.

age = 23 → Here age is the variable which stores the integer.

price = 25.99 → Here price is the variable which stores the float.

(*) The variables are stored in a MEMORY.



list = [1, 2, 3, 4, 5]
tuple = (1, 2, 3)
dict = {1: "Arijit", 2: "address"}
Set = {1, 2, 3, 4}
Here, the variables are list, tuple, dict, set

(1) Example :-

```
x = 10      # integer
y = 20.0    # float
name = "Arijit"  # string
is_active = True  # Bool
```

(A) Variable naming Rule :-

1. Variable names can be a combination of upper case or lower case letters, digits or an underscore.

```
myName = "Arijit"
anyDigit1 = 12
Variable_name = "Sourav"
```

(2) Variable name cannot start with a number.

1Variable = X [Wrong]

Variable1 = ✓ [Correct]

(iii) We cannot use special characters to make a variable name.
like → !, #, @, %, $

(iv) Variables can be any length. [len()]

(B) Dynamic Typing
Python dynamic typing is the variable data can be changed at any time.

* Ex:-

```
x = 1
print(x)
x = 2
print(2)
```

output:-
1
2

* Ex:-

```
x = 12
x = "name"
print(x)
```

output
name

* Ex:-

```
x = 1
x = 2
print(x)
```

output:-
2

* Ex:-

```
x = "name"
x = "Arijit"
print(x)
```

output
Arijit

(i) * Ex:-

```
x = 1
x = 2.0
print(x)
```

output:-
2.0

* Ex:-

```
x = 1
x = 22.0
print(x)
```

output:-
22.0

(C) (i) Multiple Assignments:-

```
a, b, c = 1, 2, 3
print(a, b, c)
```

output:-
1, 2, 3

(ii)

```
a = b = c = 1, 2, 3
print(a, b, c)
```

output:- 1 2 3

DATA TYPES

There are 5 types of data types.

- Integer
- String
- Float
- Bool
- None

① Integer Data types

The Integer data types in python represents the whole numbers (Positive, Negative, Zero) without any 'decimal or fractional part'
[Numbers starting fromm Zero]

(i) Declaring in Python

```
x = 10      # Positive integer
y = -5      # Negative integer
z = 0       # Zero
```

(ii) checking type

*
```
age = 20
print (type(age))
```
output:
<class 'int'>

*
```
x = 55
print (type (x))
```
output:
<class 'int'>

*
```
number_1 = 30
print ( type (number_1))
```
Output:
<class 'int'>

*
```
y = -5
print( print( y
print (type (y))
```
output:
<class 'int'>

(2) String Data type :-

String in Python is a sequence of characters enclosed in single (') , double (") , or triple quotes (''' or """).

(i) Declaring in Python :-

```
String_var1 = " Hellow"  # "double Quote"
String_var2 = ' Hellow'  # 'single quote'
String_var3 = ''' Hellow''' # 'triple quote'

print (type (String_var1))
print (type (String_var2))
print ( type (String_var3))
```

output :-

```
<class 'str'>
<class 'str'>
<class 'str'>
```

(ii) Multiline storing written in triple quotation.

```
String_var = ''' Arijit
das'''
print( String_var)
```

output :-

```
Arijit
das
```

(3) Float Data Type :-

A float data type in python represents decimal or real numbers, allowing fractional values.

(i) Declaring a Float :-

```
x = 3.14      # A normal float
y = -2.12     # A negative float
z = 1.0       # A float with a whole [0]

print (type (x))
print (type (y))
print (type (z))
```

Output :-

```
<class 'float'>
<class 'float'>
<class 'float'>
```

(ii) Scientific Notation :- [Exponential]

```
x = 3e4

print (x)
print (type (x))
```

output :-

```
30,000
<class 'float'>
```

A float can also be written in scientific notation using 'e' or 'E', where 'e' represents ['x 10^"]

Dry Run :-

```
x = 3e4 #  3 x 10^4
       = 3 x 10 x 10 x 10 x 10
       = 30,000
```

```
S1 = '''Hello
World'''
print(S1)
```

output

```
Hello
World
```

```
S2 = '''Hello world'''
print(S2)
```

output

```
Hello world
```

iv) Concatenation (+)

```
S1 = "Hellow"
S2 = " World"
result = S1 + " " + S2
print(result)
```

output:

```
Hellow World
```

v) Repeatation

```
String = "Hi"
print(String * 3)
```

output:

```
HiHiHi
```

```
String = "Hi\n"
print(String * 3)
```

output:

```
Hi
Hi
Hi
```

Strings

(A)

Defination :

String is a data type that stores a sequence of characters.

(B)

Indexing :-

A ri i J j t → index

[0] [1] [2] [3] [4] [5]

* Index are helps us to access characters.
* index → Position of each letter (upper/lower).
* It always starts from 0.
* **Syntax :-**

str [ ]

(1) **Example :-**

" A ri i J i t _ D a s "

0 1 2 3 4 5 6 7 8 9

→ str [0] = 'A'

→ str [8] = 'a'

(2)

```
str = "A rijit"
ch = str[0]
print(ch)
```

output :-

A

(3)

```
str = "A rijit"
ch = str[1]
print (ch)
```

output :-

ri

(4)

```
str = "A rijit"
ch = str[2]
print (ch)
```

output :-

i.

© **String slicing :-**

It helps to accessing parts of a strings.

✱ **Syntax :-**

```
str [index : index]
    (starting) : (ending)
```

✱ **Programe :-**

```
str = "A r i j i t"
      0 1 2 3 4 5
print ( str [0 : 6])
```

✱ **output :-**

A r i j i t

✱ Basically it takes the first index which is written for example ↓

str [0 : 6] → A
↓
first index

and ignores the last written index and print the previous index. for example ↓

str [0 : 5] → i
↓
last index

print only 'r and i'

```
str = "A r i j i t"
      0 1 2 3 4 5
print ( str [ 0 : 5])
```

✱ **output :-**

A r i j i

print only 'iji'

```
str = "A r i j i t"
      0 1 2 3 4 5
print ( str [2 : 5])
```

✱ **output :-**

i j i

print only 'A ri'

```
str = "A r i j i t"
      0 1 2 3 4 5
print ( str [0 : 2])
```

✱ **output :-**

A r i

print only 'jit'

```
str = "A r i j i t"
print ( str [3 : 6])
```

✱ **output :-**

j i t

print only 'Arijit'

```
str = "A r i j i t"
print ( str [ : ])
```

Another method

```
str = "Arijit"
print ( str [0 : ])
```

Output

Arijit

① Negative string slicing :-

"S t r i n g"

[-6] [-5] [-4] [-3] [-2] [-1]

→ count →

✱ Syntax :-

[-(starting index) : -(ending index)]

✱ Programm :-

Print only 'i n g'

```
Str = "S t r i n g"
print (Str[-3 :])
```

✱ output

ing

Print only 'S t r'

```
Str = "S t r i n g"
print (Str[-6 :-3])
```

✱ output

S t r

print only 'r i n'

```
Str = "S t r i n g"
print (Str[-4 :-1])
```

✱ output

r i n

print only 'S t r i'

```
Str = "S t r i n g"
print (Str[-6 :-2])
```

✱ output

S t r i

print only 't r i'

```
Str = "S t r i n g"
print (Str[-5 :-2])
```

✱ output

t r i

String Functions

1. str.capitalize() 4. str.lower()
2. str.endswith("")
3. str.find("")
4. str.count()
5. str.replace()
6. str.upper()

---

1. str.capitalize()

This method is used to write the first letter in capital.

(*) Program:

```
str = "i am Arijit"
print(str.capitalize())
```

output:

I am Arijit

2. str.endswith("")

This method is used to returns True if the string ends with the same substr given as a argument in the method. or if the substr is different then it will give false as a output.

(*) Program:

```
str = "Arijit"
print(str.endswith("it"))
```

output:

True

(*) Program:

```
str = "Arijit"
print(str.endswith("in"))
```

output:

False

⑨ str.find(" ")

This method is used to find the index no. by given the specific substr in the bracet.

✱ Program :-

```
str = "Arijit"
print (str.find("i"))
print (str.find("j"))
print (str.find("r"))
```

output :-

```
5
3
1
```

④ str.count(" ")

```
str = "I am Arijit"
print (str.count("i"))
print (str.count(" "))
print (str.count("m"))
```

output :-

```
2
2
1
```

The space(" ") is a character in python. that's why it returns 2 as a output. Because there are two space characters.

This method helps us to count the no. of characters which are present as a string.

⑤ str.replace(old, new)

This method is used to replace the one character to another character

✱ Program :-

```
Str = "Arijit is a student"
print(str.replace("student","boy"))
```

output :-

```
Arijit is a boy
```

⑥ str.upper()

This method is used to print the all characters in uppercase.

✱ Program :-

```
str = "arijit"
print(str.upper())
```

output :-

```
ARIJIT
```

(7) **str.lower()**

This method is used to print the all charecters in lower.

Program:-

```
str= "ARIJIT"
print (str.lower())
```

Output:-

arijit

(8) WAP to print a name from user & print it's length.

```
str = raw_input ("Enter your name :")
print (len (str))
print ("length of your name:", len (str))
```

Output:-

Enter your name : Arijit

6

length of your name : 6

(4) Bool data type :

The bool data type represents boolean values like, True and False. It commonly used in conditional statements and logical operations.

(i) Declaring Boolean Variables

```
x = True
y = false
print (type (x))
print ( type (y))
```

output :

```
<class 'bool'>
<class 'bool'>
```

* True is treated as = 1 | True = 1
* False is treated as = 0 | False = 0

(ii) Boolean as numbers :

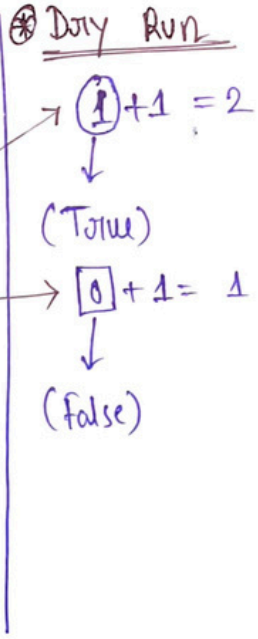
True = 1
False = 0

```
print (True + 1)
print ( false + 1)
```

output :

```
2
1
```

* Dry Run



```
print(True * 2)
print(False * 2)
print(True / 2)
print(False / 2)
```

output:

```
2
0
0.5
0.0
```

(iv) Comparison operators :

>, <, ==, !=, <=, >=

```
print (5 > 2)    # True
print (5 < 2)    # False
print (5 == 2)   # False
print (5 >= 2)   # True
print (5 <= 2)   # False
```

Types of operators

(*) There are four types of operators.

i) Arithmatic operator [+, -, *, %, /, **]

ii) Relational / comparison operator [==, >, <, >=, <=, !=]

iii) Assignment operator [=, +=, -=, /=, %=, **=]

iv) Logical operator [not, and, or]

i) ARITHMATIC OPERATOR :-

(a) a = 5 + 3 # output : 8 [Addition (+)]

(b) a = 10 - 4 # output : 6 [Subtraction (-)]

(c) a = 6 * 7 # output : 42 [multiplication (*)]

(d) a = 10/2 # output : 5 [Division (/)]

(e) a = 10 // 3 # output : 3 [Floor division (//)]

(f) a = 10 % 3 # output : 1 [Modulus (%)]

(g) a = 2 ** 3 # output : 8 (2^3) [Exponantial (**)]

important :

(i) Floor division (//) → Devides and rounds down the nearest integer.

(ii) Modulus (%) → Returns the remainder of a division.

(iii) Exponantial (**) → Raises a number to the power of another.

ii) Relational Operator : [==, >, <, >=, <=, !=]

- == → Equal to
- > → Greator than
- < → less than
- >= → Greater than equal to
- <= → less than equal to
- != → Not equal to

Assignment operators:

- `=` → used for store value of any data type
- `+=` → ex: →

```
a = 10
a += 10
print(a)
```

output: 20

- `-=` → ex: →

```
a = 10
a -= 10
print(a)
```

output: 0

- `*=` → ex: →

```
a = 10
a *= 10
print(a)
```

output: 100

- `/=` → ex:-

```
a = 10
a /= 2
print(a)
```

output: 5.0

Logical operators:

- **and** = Returns always boolean value [T/F]

```
a = 10
b = 20
print(a < b and b > a)
```

It returns True if two conditions are true. ## True

- **OR** = It returns True if only one condition is true between two

```
a = 10
b = 20
print(a < b or b < a)
```

True

- **NOT**

~~a = [1,2,3]~~
~~c = b~~
~~b = [1,2,3]~~
print

```
x = 5
if not x > 10:
    print("True")
```

True

Taking user as Input :

We can usually take input as a integer, float, string and bool. But the compiler take the input usually as a string by default. We can take input by writting only → `input()` function.

✱ Here's why...

First prog

①
```
a = input("Enter first num :")
b = input("Enter second num :")
Sum = a + b
print(f" Sum : ", Sum)
```

output:

Enter first num : 14

Enter second num : 12

Sum : 1412

Reason :

Why this happen??

This happens when we does not specifically write "int, float, str". It returns a string character (concatination (1412)

②
```
a = int(input("Enter first num:"))
b = int(input("Enter second num :"))
Sum = a + b
print(f" Sum :", Sum)
```

output:

Enter first num : 14

Enter second num : 12

Sum : 26

Reason II

Now we specifically write 'int', so it returns an integer number after sum

Taking integer as a input

First Prog:

①
```
a = input("Enter first num:")
b = input("Enter Second num:")
print(type(a))
print(type(b))
```

Output:

```
<class 'str'>
<class 'str'>
```

①
```
a = int(input("Enter first num:"))
b = int(input("Enter first num:"))
print(type(a))
print(type(b))
```

Output:

```
<class 'int'>
<class 'int'>
```

Taking String as a input:-

```
name = str(input("Enter your name:"))
print(f"name : {name}!")
```

output:

```
name: Enter your name : Arijit
      name : Arijit !
```

Taking float as a input:

```
a = float( input(" Enter your height :"))
print ( f" your height is : {a} !")
```

Output :

```
Enter your height : 6.5
Your height is : 6.5 !
```

Taking bool as a input:

- use "yes"/"no"
- strip().lower()

. strip()

. strip() function used for removing the leading space between the characters

eg →

Program :

① WAP a program to write "yes" as "True" and "No" as "False".

```
name = str (input (" Enter yes for \" True \" and No for \"False\"))
       . strip() . lower() == " yes "

print( name)
```

output:

```
Enter yes for " True " and No for "false" : yes
True
```

② WAP to print 1 for True and 0 for false.

```
num = int(input("Enter 1 for \"True and 0 for \"false\":"))
      .strip().lower() == '1'

print(num)
```

output:

Enter 1 for "True" and 0 for "false": 1

True

How it works :-

- input() takes the user input
- .strip().lower() removes spaces and turns the letters into lower case.
- == "1" comparison returns True if the input is "yes", otherwise false.

Type casting and type conversion

There are two type of data type conversion in python.

(A) → Implicit Type casting [Type conversion]
(B) → Explicit Type casting

(A) Implicit Type casting :- (i) int → float

That type of type casting are done by the compiler.

(*) eg →

~~int = 12~~
~~float = 13.5~~
~~x~~

```
a = 12
b = 13.5
x = a+b
print(x)
```

output:
25.5

That is called implicit type casting. Here a is a integer variable and b is a float variable. After adding (int + float), the compiler returns a float value in return.

Basically,
Implicit type casting are done by the compiler.

```
print(type(a)) # <class 'int'>
print(type(b)) # <class 'float'>
print(type(x)) # <class 'float'>
```

(ii) int → complex

```
a = 13
b = 3 + 2J
x = a + b
print(x)
```

output:
16 + 2J

```
print(type(a)) # 'int'
print(type(b)) # 'complex'
print(type(x)) # 'complex'
```

(iii) Boolean (bool) ⟶ Integer (int)

Python consider bool as True and false.

True = 1

False = 0

eg⟹

```
a = True
b = 5
x = a + b
print (x)
```

```
print (type (a))   # <class 'bool'>
print ( type (b))  # <class 'int'>
print (type (x))   # <class 'int'>
```

Output:

6

(iv) Characters (str) is not implicitly converted to numbers.

```
a = "100"
b = a + 90
print(b)
```

Output:

#TypeError: cannot add str + int.

(B) Explicit Type Casting:

Explicit type casting in python is also called "manual type casting".

i)
```
# float to int
a = 10.0
print(int(a))
print(type(a))
```

- Here, 'a' is a float variable, but we can convert it into an integer manually.
- → At first the a variable type was <class 'float'>
- → After the conversion it is <class 'int'>

output:

```
10
<class 'int'>
```

ii)
```
# int to float
a = 10
print(float(a))
print(type(float(a)))
```

output

```
10.0
<class 'float'>
```

iii)
```
# int to bool
a = 1
print(bool(a))
print(type(bool(a)))
```

output:

```
True
<class 'bool'>
```

```
a = 0
print(bool(a))
print(type(bool(a)))
```

output:

```
False
<class 'bool'>
```

(iv) Bool to integer

```
a = True
print( int(a))
print( type ( int (a)))
```

output

1

<class ' int'>

```
a = False
print ( int(a))
print ( type ( int(a)))
```

output

0

<class ' int'>

(v) float to bool

```
a = 1.0
print( bool (a))
print( type ( bool(a)))
```

output

True

<class' bool'>

```
b = 0.0
print ( bool (b))
print ( type ( bool (b)))
```

output

false

<class 'bool'>

(vi) bool to float

```
a = True
print ( float ( a))
```

output

1.0

```
a = False
print ( float ( a))
```

output

0.0

(xii) int to storing

```
a = 12
print ( str(a))
print ( type ( str(a)))
```

output

12

<class ' str'>

(xiii) Storing to int

```
a = "12"
print ( int(a))
print ( type ( int(a)))
```

12

~~<class 'str'>~~

<class ' int'>

(xiv) str to float

```
a = "1"
print(float(a))
```

output

1.0

(xv) float to str

```
b = 1.0
print(str(b))
```

output

1.0

(xvi) str to bool

```
a = "1"
print(bool(a))
```

output

True

(xvi) bool to str

```
a = True
print(str(a))
print(type(str(a)))
```

output

True

<class 'str'>

Conditional statements

① If statement:

```
if condition:
    statement
```
→ syntax

(i) WAP to check the number is greater than 10 or not.

```
num = int(input("Enter a number : "))
if num > 10:
    print("number is greater than 10")
print("Program ended")
```

Output:

```
Enter a number : 15
number is greater than 10
Program ended
```

(ii) Exam passing condition using if statement.

```
marks = int(input("Enter your marks : "))
if (marks > 21):
    print ("Pass")
```

Output:

```
Enter your marks : 25

Pass
```

③ Check the number is odd or Even.

```
num = 10
if (num % 2 == 0)
    print ("Even")
```

output:

Even

④ WAP to check a user defined string length is greater than 10 or not.

```
Str = input ("Enter a string:")
x = len (Str)
print ("length:", x)
if x > 10 :
    print (" string length is greater than 10.")
```

output:

Enter a string: Arijit Das_
length : 11
string length is greater than 10.

Conditionals Statements :

There are three (3) types of conditional statements in Python.

(1) → If
(2) → If ~~else~~ else
(3) → if - elif - else
(4) → Nested if

(2) IF-Else

(1) WAP a program if your age greater than 18 then you can drive but if your age is not greater than 18 than print you can't.

```
age = int(input(" Enter your age :"))
if (age > 18):
    print ("You can drive")
else:
    print ("you cannot drive")
```

If-else :- [SYNTAX]

```
if (condition):
    statement
else:
    statement
```

Output:-

```
Enter your age : 19
you can drive
```

(2) WAP a program if it rain today then I will not go to School or I will.

```
rain_today = str(input(" Enter (Yes/No) if it rain tody : ").strip().lower())
if rain_today == "yes":
    print (" I will not go to School")
else:
    print(" I will go to School")
```

Output :-

Enter (Yes/No) if it rain today : yes

I will not go to School.

(*) **strip()**

We use strip() to remove the leading or trailing spaces between the character.

(*) Programme :-

```
myName = "  Arijit  "
print( myName . strip())
```

Output

Arijit

(*) **lower ()**

We use the lower() to print the characters in lower.

(*) Programme :-

```
myName = "ARIJIT"
print( myName . lower())
```

Output

arijit

(*) **strip() . lower ()**

We use this together to make the remaining ~~value~~ conditional statement same.

```
if rain_today == "yes"
```

By using strip() . lower() ,

if the storing "yes" written as " yes" or "yes" or "yes" , it will remain same as "yes".

③ WAP to check the no. odd or even.

```
age = int( input(" Enter a number as age :"))
if  age % 2 == 0 :
    print( "Even")
else :
    print ( " odd")
```

Output:

```
Enter a number as age : 18
Even
```

④ WAP if your marks greater than equals to 21 then print "PASS" else print "FAIL".

```
marks = int (input ( "Enter your marks : "))
if  marks >= 21 :
    print ( " PASS")
else :
    print ( "FAIL")
```

output:

```
Enter your marks : 70
PASS
```

If-elif-else:

Syntax:

```
if (condition):
    statement
elif (condition):
    statement*
else:
    statement
```

① Check whether a number is positive, negative or zero.

```
num = int(input("Enter a num:"))
if (num > 0)
    print("Positive")
elif (num < 0)
    print("Negative")
else:
    print("Zero")
```

Output:

Enter a num: 18

Positive

② WAP to print the marks of your exam.

marks >= 90 , marks <= 100 , grade 'A'

marks >= 80 , marks <= 89 , grade 'B'

marks >= 70, marks <= 79 , grade 'C'

else, print("D")

```
marks = int(input("Enter your marks : "))
if marks >= 90 and marks <= 100:
    print("A")
elif marks >= 80 and marks <= 89:
    print("B")
elif marks >= 70 and marks <= 79:
    print("C")
else:
    print("D")
```

Output:

```
Enter your marks : 75
C
```

(3) WAP to find the greatest of 3 numbers entered by the user.

```
a = int(input("Enter first number : "))
b = int(input("Enter second number : "))
c = int(input("Enter third number : "))
if a > b and a > c:
    print("greater number is:", a)
elif b > a and b > c:
    print("greater number is : ", b)
else:
    print("greater number is:", c)
```

output:

Enter first number: 12

Enter second number: 13

Enter third number: 95

Greater number is: 95

(14) WAP to check the letter is A or not.

```
letter = "A"
if letter == "B"
    print("B")
elif letter == "C"
    print("C")
elif letter == "A"
    print("letter is A.")
else:
    print("letter is not A,B,C")
```

output:

letter is A.

⑥ WAP to check a letter is vowel or not.

```
letter = str(input("Enter a letter : ")).strip().lower()
if letter == 'a':
    print("This is a vowel")
elif letter == 'e':
    print("This is a vowel")
elif letter == 'i':
    print("This is a vowel")
elif letter == 'o':
    print("This is a vowel")
elif letter == 'u':
    print("This is a vowel")
else:
    print("This is not a vowel")
```

Output:

```
Enter a letter : a
This is a vowel
```

DICTIONARY and DICTIONARY METHODS :-

Definition :

→ Dictionary is a method in python to store the data in ~~im~~mutable Way.

→ This is ~~im~~mutable (~~not~~ changeable).

Syntax

```
Variable_name = {
                Key : Value,
}
```

- This is a set up of Key : Value pairs.
- It can store string, integer, float, List, tuple data type Variable as Key : Value
- Mutable (changable)
- List and Tuple are not data type

Programme - 1 :

Key : Value

```
diet = {
        int : 18,
        float : 18.0,
        str : "Arijit",
        list : [18, 9, 10],
        Tup : (1, 2, 3),
      }
print (diet)
```

Here the first three data types and the list, tuple methods can be store in a single dictionary.

WAP in python to show all data types as: Key : Value

```
my_diet = {
        "Integer" : 12,
        "float_Var" : 12.0,
        "string" : "Arijit Das",
        "list_Var" : [12, 0, 9],
        "Tup" : (1, 2, 4),
}
print (my_diet)
```

WAP on python to show your BIO-DATA.

```
print ("BIO-DATA")
diet = {
        "Name" : "Arijit Das",
        "Age" : 18,
        "Address": "Dum Dum",
        "father's Name" : "DeepanKar Das",
        "Mother's Name": "Rupali Das",
        "Education" : " Student",
    }
print (diet)
```

*	You can use another dictionary variable name also.
*	It should be written as the topic as given in problem.

WAP in python to show your mark sheet.

```
print("Mark sheet")
    diet = {
            "Subject-1" : 90,
            "Subject-2" : 90,
            "Subject-3" : 100,
            "Subject-4" : 80,
            "Subject-5" : 90,
        }
print ( diet)
```

* Dictionary Key name should be different or it will occure a error.
* All elements in the dictionary are ~~im~~mutable./ changable.
* ~~Im~~mutable means ~~not~~ changable.

Built in functions in Python dictionary.

① len() → Returns the numbers of Key:Value pairs.

Program

(i)

```
dict = {
        1: "Arijit",
        2: 18,
        3: [1,2,3],
        4: (12, 13, 14),
      }
print (len (dict))
```

output:

4

(*) → Here the no. of Key-Values are 4 so the length of dictionary is 4.

② Sorted() → (i) For integer Key, it sort the keys numberwise.
(ii) For string Key, it sort the keys alphabatically.
(*) It only sort the keys not the values.

(*) Integer:

```
dict = {
        1: "Arijit",
        2: 18,
        3: [1,2,3],
        5: [4,5,6],
        4: (1,2,3),
      }
print (Sorted(dict))
```

output:

[1, 2, 3, 4, 5]

(*) String:-

```
dict = {
        "name": "Arijit",
        "Age": 18,
        "Marks": [95, 96, 97],
        "course": "BCA",
      }
print(Sorted(dict))
```

output:

['Age', 'Marks', 'course', 'name']

(*) In that case 'course' Key should be in the 2nd but the system give importance to the Upper case letter first, so the 'M' is in the 2nd position after 'A'. [M=Marks, A=Age]

```
dict = {
        "name" : "Avijit",
        "age" : 18
        "course": 'BCA',
        "marks": [ 12, 13, 14],
      }
print(dict)
print ( sorted (dict))
```

output

```
[a, c ['age' , 'course', 'marks', ' name']
```

```
dict ={
        "name" : "Avijit" ,
        "address": "Dum Dum",
        "course": "BCA",
        "subjects" :"Python",
        "age" : 18 ,
      }
print ( sorted ( dict))
```

output

```
[ "address", "age" , " course", " name" , " subjects" ]
```

DICTIONARY METHODS()

① clear() → Removes all the element from the dict.

i)
```
dict = {
        1 : "name",
        2 : "Age",
        3 : "value",
        4 : "Add",
      }
dict.clear()
print (dict)
```

output:
{}

ii)
```
dict = {
        "name" : "Avijit",
        "Age" : 18,
        "address" : "Dum Dum",
      }
dict.clear()
print(dict)
```

output:
{}

② copy() → Returns a shallow copy of the dictionary.

i)
```
dict = {
        1 : "Avijit",
        2 : "Alice",
        3 : "John",
        4 : "Adam",
      }
x = dict.copy()
print(dict)
print (x)
```

output:
{1 : 'Avijit', 2 : 'Alice', 3 : 'John', 4 : 'Adam'}

ii)
```
dict = {
        1 : 18,
        2 : 19,
        3 : 20,
        4 : 50,
      }
print (dict.copy())
```

output:
{1 : 18, 2 : 19, 3 : 20, 4 : 50}

3) .Keys() → Returns all the Keys.

i)
```
dict = {
        1 : "Arijit",
        2: "Adam",
        3: "Alice",
        4: "John",
}
print (dict.keys())
```

output:

dict_keys([1,2,3,4])

ii)
```
dict = {
        "name" : "Arijit",
        "Age" : 18,
        "Address": "Dum Dum",
}
print (dict.keys())
```

output:

dict_keys (["name","Age','Address'])

4) .Values() → Returns all the Values.

i)
```
dict = {
        1: "Arijit",
        2: "Adam",
        3: "Alice",
        4: "John",
}
print (dict.values())
```

output:

dict_values (["Arijit','Adam','Alice','John'])

ii)
```
dict = {
        "name" : "Arijit",
        "Age" : 18,
        "Address": "Dum Dum",
}
print (dict.values())
```

output:

dict_values (['Arijit', 18, 'Dum Dum'])

(b) fromKeys() → Creates dictionary using this method

variable_name = dict.fromKeys(K, V)
K = Keys, V = Values

(i) You have to store the all keys in a list.

(ii) Then you have to store the values for the given keys.

(iii) It can be created by the help of list.

Parameters :
(i) List of Keys [Any variable]
(ii) Values [Any variable]

program :→ (1)

(i)
```
K = ["Math", "Eng", "Physics", "Chem"]
V = 90
di = dict.fromkeys(K, V)
print(di)
```

output:

~~{'Math': 0, 'ph~~ {'Math': 90, 'Eng': 90, 'Physics': 90, 'Chem': 90}

NOTE
Key_Variable = [ ]
Value_Variable =
Variable = dict.fromkeys(K,V)
↑
(dictionary_variable)

(ii)
```
K = ['a', 'b', 'c']
V = 0
dictionary = dict.fromkeys(K, V)
print ~~(K, V)~~ (dictionary)
```

output:
{'a': 0, 'b': 0, 'c': 0}

(iii) Using (none) as default value.

```
Keys = ["a", "b", "c"]
dict = dict.fromkeys(Keys,)
print(dict)
```

When we don't take the values for the key list then the compiler takes the value 'none' automatically

output:
{'a': None, 'b': None, 'c': None}

(iv) Creating an empty dictionary using fromKeys()

```
Keys = ["a", "b", "c"]
dict = dict.fromkeys (Keys, " ")
print (dict)
```

Output:

{'a':" ", 'b':" ", 'c':" "}

(v)

```
K = [1, 2, 3]
dict = dict.fromkeys (K, )
print (dict)
```

Output:

{1: None, 2: None, 3: None}

(vi)

```
K = [1, 2, 3]
dict = dict.fromKeys (K, " ")
```

Output:

{1:" ", 2:" ", 3:" "}

Also we can take the Keys and Values by passing arguments in the fromKeys() method

- Syntax → dict.fromKeys (Keys, Values) → by passing two arguments

```
dict = dict.fromkeys ([1,2,3], "arijit")
```

output:

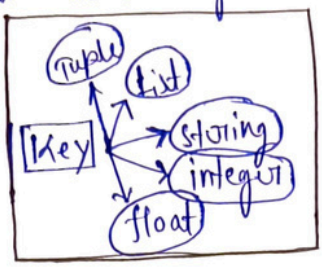
{1: "arijit", 2: "arijit", 3: "arijit"}

⑥ .get()

.get() method helps us to get the specific values by the key

Syntax:

```
dict.get("Key")
```



(i)

```
dict={
    1:"Arijit",
    2:"Sourav",
    3:"Ankit",
}
print(dict.get(1))
```

output:

Arijit

```
dict={
    1:"Arijit",
    2:"Sourav",
    3:"Ankit",
}
print(dict.get(2))
```

output:

Sourav

```
dict={
    1:"Arijit",
    2:"Sourav",
    3:"Ankit",
}
print(dict.get(3))
```

output:

Ankit

if we print a key that is not present at the dictionary then it will occur 'none'.

```
dict={
    1:"Arijit",
    2:"Sourav",
    3:"Ankit",
}
print(dict.get(4))
```

Key-4 is not present at the dictionary, so it print 'None'.

output:

None

⑦ .items()

.items() method returns all the elements which is present at dictionary.

Syntax:-

```
dict.items()
```

①
```
dict = {
        "name": "Arijit",
        "Age" : 19,
        "ID": 231001102245,
        "college": "TIU",
       }
print( dict.items())
```

Output:

```
dict_items ([('Name', 'Arijit'), ('Age', '19'), ('ID', '231001102245'),
            ('college', 'TIU')])
```

②
```
diet = {
        1: 19,
        2: 20,
        3: 21,
        4: 22,
        5: 23,
}
print (dict.items())
```

Output:

```
diet_items([('1', '19'), ('2', '20'), ('3', '21'), ('4', '22'), ('5', '23')])
```